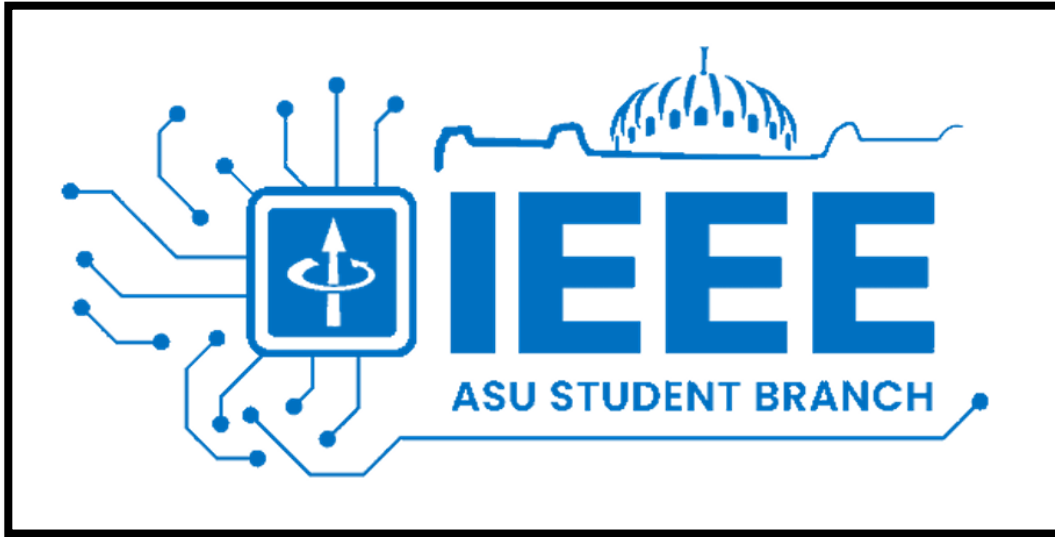




=====IEEE – Computer Arithmetic Workshop=====

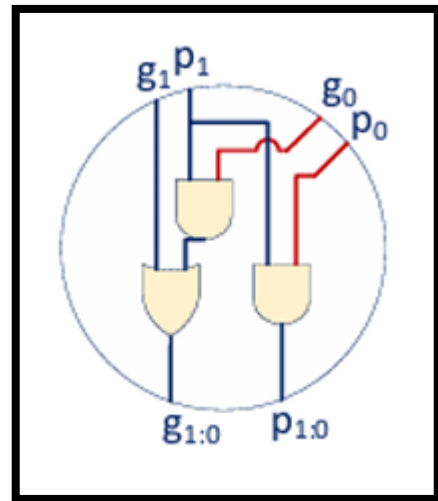
=====Assignment(2)=====

> 64-bit Prefix Adder <

[Brent Kung Topology]

Name: Ibrahim Mohamed Mohamed Hanafy

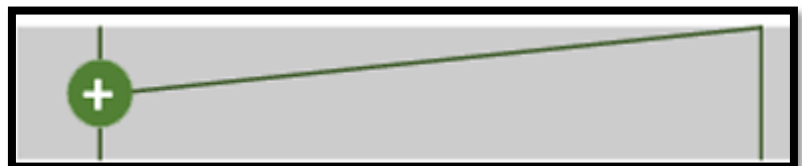
C Operation



-RTL:

```
352 module C_operation(input gli,pli, gri,pri, output gl,pl);
353     assign gl = gli | (pli & gri);
354     assign pl = (pli & pri);
355
356 endmodule
```

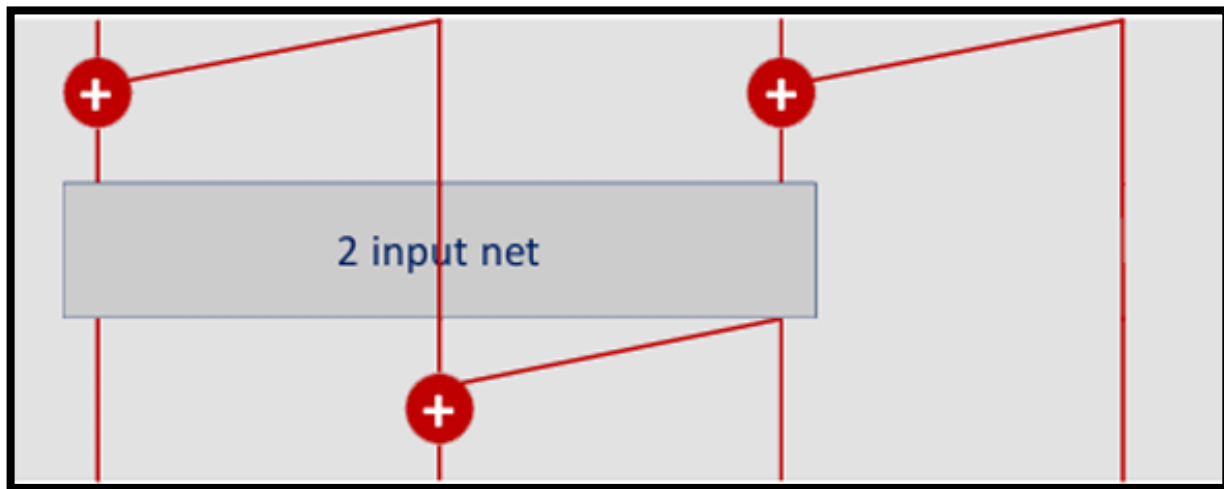
2-input net



-RTL:

```
341 module i2_input_net (input [1:0] gin, pin, output [1:0] gout, pout);
342
343     C_operation C_0 (gin[1], pin[1], gin[0], pin[0], gout[1], pout[1]);
344
345     assign gout[0] = gin[0];
346     assign pout[0] = pin[0];
347 endmodule
348
```

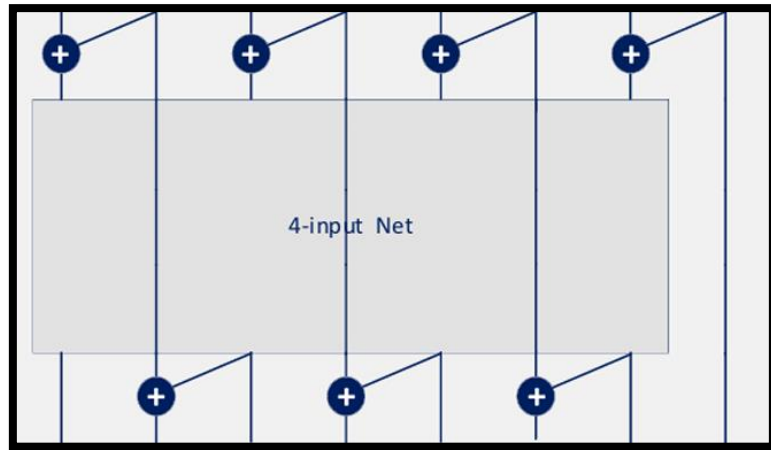
4-input net



-RTL:

```
308 module i4_input_net (input [3:0] gin, pin , output [3:0] gout, pout);
309
310     wire [1:0] g_odd, p_odd, g_odd_next, p_odd_next;
311     wire [1:0] g_even, p_even, g_even_next, p_even_next;
312
313     wire [1:0] g_out_odd, p_out_odd;
314     //0. Split Odds and evens=====
315     assign g_odd = {gin[3], gin[1]};
316     assign p_odd = {pin[3], pin[1]};
317     assign g_even = {gin[2], gin[0]};
318     assign p_even = {pin[2], pin[0]};
319     //1. [Work on Odds]======(C Operation)=====(4/2 loops)=====
320
321     C_operation C_1 (g_odd[0], p_odd[0], g_even[0], p_even[0], g_odd_next[0], p_odd_next[0]);
322     C_operation C_2 (g_odd[1], p_odd[1], g_even[1], p_even[1], g_odd_next[1], p_odd_next[1]);
323
324     //2. [Pass Ready Odds to layer below]---->[now Odds are ready for Out]=====
325
326     i2_input_net net_2(g_odd_next, p_odd_next, g_out_odd, p_out_odd);
327
328     //3. [Work on Evens]======(C Operation)======(4/2 - 1 Loops)=====
329
330     C_operation C_3 (g_even[1], p_even[1], g_out_odd[0], p_out_odd[0], g_even_next[1], p_even_next[1]);
331
332
333     assign gout = {g_out_odd[1], g_even_next[1], g_out_odd[0], g_even[0]};
334     assign pout = {p_out_odd[1], p_even_next[1], p_out_odd[0], p_even[0]};
335
336 endmodule
```

8-input net



-RTL:

```

241 module i8_input_net (input [7:0] gin, pin , output [7:0] gout, pout);
242
243     wire [3:0] g_odd, p_odd, g_odd_next, p_odd_next;
244     wire [3:0] g_even, p_even, g_even_next, p_even_next;
245
246     wire [3:0] g_out_odd, p_out_odd;
247 //0. Split Odds and evens=====
248     genvar i;
249     generate
250         for (i = 0; i < 4; i = i + 1) begin : split_odd_even
251             assign g_even[i] = gin[2*i];
252             assign p_even[i] = pin[2*i];
253
254             assign g_odd[i] = gin[2*i + 1];
255             assign p_odd[i] = pin[2*i + 1];
256         end
257     endgenerate
258
259
260 //1. [Work on Odds]======(C Operation)===(8/2 loops)=====
261
262     generate
263         for (i = 0; i < 4; i = i + 1) begin : odd_c_ops
264             C_operation C_odd (
265                 g_odd[i], p_odd[i],
266                 g_even[i], p_even[i],
267                 g_odd_next[i], p_odd_next[i]
268             );
269         end
270     endgenerate

```

```

273 //2. [Pass Ready Odds to layer below]---->[now Odds are ready for Out]====
274
275     i4_input_net net_3(g_odd_next, p_odd_next, g_out_odd, p_out_odd);
276
277 //3. [Work on Evens]======(C Operation)======(8/2 - 1 Loops)=====
278
279     generate
280         for (i = 1; i < 4; i = i + 1) begin : even_c_ops
281             C_operation C_even (
282                 g_even[i], p_even[i],
283                 g_out_odd[i-1], p_out_odd[i-1],
284                 g_even_next[i], p_even_next[i]
285             );
286         end
287     endgenerate
288
289 //4. Reassemble everything=====
290     assign g_even_next[0] = g_even[0];
291     assign p_even_next[0] = p_even[0];
292
293     generate
294         for (i = 0; i < 4; i = i + 1) begin : reassemble
295             assign gout[2*i] = g_even_next[i];
296             assign pout[2*i] = p_even_next[i];
297
298             assign gout[2*i + 1] = g_out_odd[i];
299             assign pout[2*i + 1] = p_out_odd[i];
300         end
301     endgenerate
302 endmodule

```

16-input net

32-input net

64-input net

Can be written as the 8-input net where each one instantiates the previous one.

Final 64 Adder

-RTL:

```
1  module Brent_Kung_64bit_Adder
2      (
3          input [63:0] A,B,
4          input Cin,
5          output [63:0] S,
6          output Cout
7      );
8
9  //1.G, P Generation Logic=====
10     wire [63:0] g, p;
11     assign g = A & B;
12     assign p = A ^ B;
13 //=====
14
15 //2.Carry Network=====
16     wire [63:0] C;
17     wire [63:0] gout, pout;
18     i64_input_net top_net(g,p, gout,pout);
19
20     assign C[0] = Cin;
21     genvar i;
22     generate
23         for (i = 0; i < 63; i = i + 1) begin : carry_eval
24             assign C[i+1] = gout[i] | (pout[i] & Cin);
25         end
26     endgenerate
27
28     assign Cout = gout[63] | (pout[63] & Cin);
29
30 //=====
31
32 //3.SUM Evaluation=====
33     assign S = C ^ p; //this easy :)
34 //=====
35
36
37 endmodule
```

Testbench *[100 Random Tests]*

```
1 module Prefix_Adder_64bit_tb;
2
3     reg [63:0] A,B; //Inputs are regs
4     reg cin;
5
6     wire [63:0] sum; //Outputs are wires
7     wire cout;
8
9     //Instantiate
10    Brent_Kung_64bit_Adder Brent_Kung_tester (A, B, cin,sum, cout);
11
12    //Task: validation of output
13    task validate_output;
14    |   if({cout,sum} != A + B + cin) $display("Error: Adding A + B + cin, time: %0t", $time);
15    endtask
16
17
18    initial
19    begin
20        A=0; B=0; //initial state
21        cin =0;
22        #10
23
24        repeat(10) //100 randomized tests
25        begin
26            A = {$random, $random}; //$random gives 32 bit only
27            B = {$random, $random};
28            cin = $random % 2; // ensure 0/1
29
30            #5; //wait for output
31
32            validate_output();
33        end
34
35        $display("Test case 1:"); //Test 1
36        A = 64'd41; B = 64'd26; cin = 0;
37
38        #10 //wait for out and monitor
39        if(sum == 64'd67 && (!cout) ) $display("Test 1 pass, time: %0t",$time);
40        else $display("Test 1 fail");
41
42
43        #10
44        $stop;
45    end
46
47    initial //monitor i/o
48    begin // this event is in the postponed event region
49        $monitor("Time=%0t | A=%b | B=%b | cin =%b | Sum=%b | Cout=%b",
50        $time,A,B,cin,sum,cout);
51    end
52
53 endmodule
```

-Waveforms , Results

		Msgs							
A	64d41	0	1303006099158949505	510696936329962597	4261515162043665725	16372595406958724293	9886199506036120463	16341960806660327772	
B	64d26	0	9548992447060661859	9887461744194085633	8562565468134438796	15353847184146560997	8732596169457702606	160368001178544656557	
cin	1h0								
sum	64d67	0	10851998546219611365	14994431080524048231	1282408060178104522	13279698517395733675	172051601784271453	13932016850805432714	
cout	1h0								

[illegible]

	Mips								
A	64d207894...	658868...	5316039298284203012	7831961897697289837	5373763667882311174	1573972301104294280	10906764121189377032	10247817928165910406	1328273866443436951
B	64d168558...	438243...	12445635880704602587	13494440856617961930	176936778933851639	1476293546218619949	1027178180850984701	17092796220836611645	6733831942830945118
cin	1'h0								
sum	64d48040...	109711...	172616787898805600	2879658680605700151	5550200396816207814	17216016577722914230	11933942302040351234	889337007592920435	156932653355575153
cout	1'h1								

[illegible]

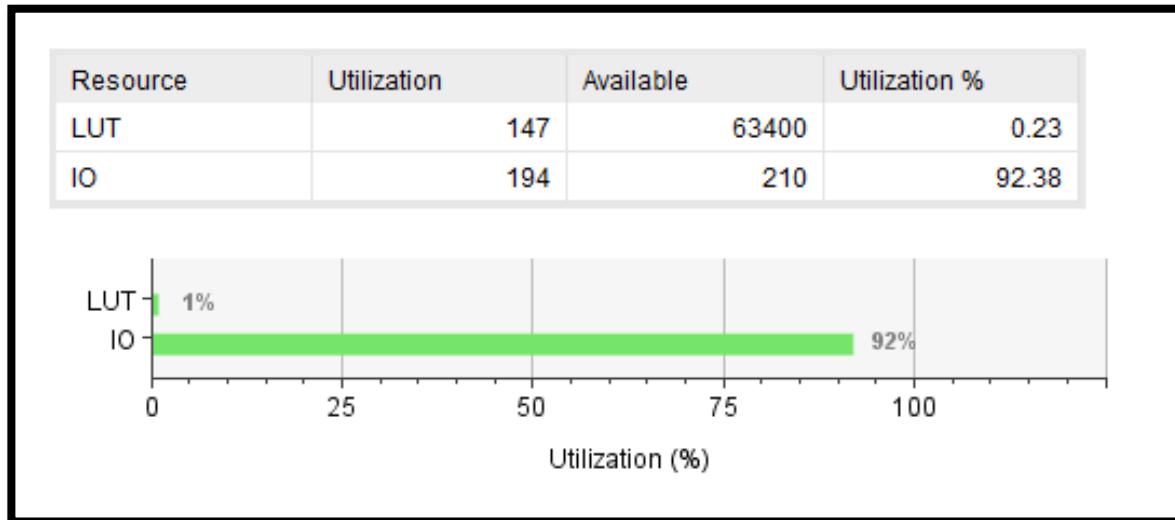
-No Errors

```
# Time=505 | A=101110111011
# Test case 1:
# Time=510 | A=00000000000000
# Test 1 pass, time: 520
++ Note: Caterpillar: E+(15
```

-Pass

FPGA Implementation [xc7a100tcsg324-1]

-Utilization Report



-Timing Report

Setup	
Worst Negative Slack (WNS):	-9.368 ns
Total Negative Slack (TNS):	-555.896 ns
Number of Failing Endpoints:	65
Total Number of Endpoints:	65

Given: 5ns max delay $\therefore$ Critical Delay = 14.368ns

Report on limitations

-This RTL Code can implement only 64,32,16,8,4,2 bit adders with a little work needed to implement them

-However for other n bit adder more RTL tweaks would be needed

